Chapter_001

Tuesday, August 19, 2025 17:51

A

Jord-01ff.fm Page 1 Wednesday, July 24, 2002 8:20 PM

CHAPTER 1

Parallel Machines and Computations

In the quest for ever-faster computers, the technique of reducing execution time by doing several things at once is an obvious way to gain speed. The surprising thing is not that parallel computers and computation are attracting increased attention, but that so much of computing science and technology has revolved around a sequential, one-thing-at-a-time, style of algorithm specification, programming, and machine design. Parallel processing has evolved underneath an overall sequential world view, and only after considerable time has it emerged as a high-level principle to guide the design of algorithms, machines, and programming languages.

The subject of parallel processing is really a topic that integrates the needs of algorithms with the capabilities of architectures. The study of parallel architectures is sterile without the assurance that the parallel computation capabilities of the architecture can actually be used to speed up the execution of a sufficiently large set of real computations. In the same way, the study of algorithms to determine the operations that can potentially be executed in parallel is unexciting if no implementable architecture can be envisioned that can actually execute them in parallel. Parallel processing is thus a study that integrates parallel algorithms and architectures by way of parallel programming languages. Therefore, in what follows we do not attempt an overall organization that separates parallel processing into architectures, languages, and algorithms. Instead, we will move between these aspects of parallel computation in an attempt to underline the importance of their interrelationships.

It is useful to keep some general questions in mind while reading this chapter. What is the nature of the multiple computational units used to do operations in parallel? How can one write recipes that specify operations to be done in parallel and others to be done sequentially? What aspects of a particular algorithm for doing a computation influence what can and cannot be done in parallel?

1

Jord-01ff.fm Page 2 Wednesday, July 24, 2002 8:20 PM

1.1 The Evolution of Parallel Architectures

Several different forms of parallelism have been pursued to solve problems faster by doing several things at once. The main classes can be described by relating them to the standard sequential computational model. In this model, one operation: add, subtract, multiply, divide, or conditional branch, can be performed at a time. The first major division of kinds of parallelism is based on parallelism over multiple data items versus parallelism over multiple operations. There is probably a better understanding of the numerical analysis topic of transformation of data through a fixed set of operations than there is of flow of control transformations, which has a shorter history in the computer science field. Thus, the simplest step from sequential programs into the field of parallelism is probably to leave the control flow of the program alone and to do multiple operations on disjoint data items simultaneously. Flynn [110] characterized the concurrency of data operations with respect to instruction streams by dividing architectures into four categories based on the number of instruction streams and the number of data streams. Single (SI) or multiple instruction (MI) streams may be combined with single (SD) or multiple data (MD) streams to form an architectural category. Thus, the original von Neumann architecture is classified as SISD (single instruction stream, single data stream), vector computers are SIMD, because a single instruction stream directs operations on multiple data, and a multiprocessor is an MIMD computer. The MISD combination is not very useful, although there are a few research machines that it might fit.

Let us briefly describe the progress toward gaining computational speed from the computer architecture and system perspective.

1.1.1 Parallelism in Sequential Computers

The original von Neumann architecture, classified as SISD, performed computation by breaking them into a one-at-a-time sequence of machine operations. Figure 1-1 shows the major components of this early architecture, namely, the control unit (CU) and the arithmetic logic unit (ALU) together forming the central processing unit (CPU) and the main memory (M). The intent here is not to show detailed implementation but the high-level description of the operations as they lead to the discussion of parallelism. Upon start, the computer enters the fetch/execute cycle of Figure 1-2 until it is halted.

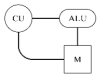

Figure 1-1 A typical von Neumann architecture.

Jord-01ff.fm Page 3 Wednesday, July 24, 2002 8:20 PM

1. Instruction fetch
2. Instruction decode and increment the program counter
3. Effective operand address calculation
4. Operand fetch
5. Execute
6. Store result

Figure 1-2 A simple fetch/execute cycle for a von Neumann machine.

Much improvement has been done to this early architecture to speed up the operations of the SISD computers. Interrupts have provided overlap between CPU operations and slow peripheral devices, input/output (I/O) processors provide concurrency between fast I/O devices and CPU while enabling the CPU and I/O processors to access the main memory directly, as shown in Figure 1-3. Multiplexing the CPU between programs in a multiprogrammed system has reduced the CPU idle time and maximized the system throughput and CPU utilization. Interleaving execution of several programs in this way led operating systems to have to manage concurrent processes, each at a different stage of its execution. High-speed block transfer between main and secondary memory made possible by I/O processors and the principle of locality have led to virtual memory management, where applications much larger than the size of the system's main memory can execute to completion. Combination of virtual memory and multiprogramming has resulted in the ubiquitous multiuser, time-sharing computing environments.

Pipelining is a powerful technique to overlap operations and introduce parallelism into a computer architecture. Through some additional hardware, the steps in the fetch/execute cycle of Figure 1-2 can be overlapped. Figure 1-4 shows a possible pipelined fetch/execute cycle. While instructions are still being executed sequentially by going through the different stages of the pipeline in the order they are fetched, several of them are being processed at different stages of the pipeline simultaneously. In this example, it takes four time units to complete the first instruction (pipeline start-up time). However, after this initial time, one instruction completes at every pipeline step. Figure 1-5 compares the non-pipelined fetch/execute cycle with the pipelined one to show the effectiveness of pipelining. There are, however, complexities associated with pipelining. For example, when a conditional jump instruction is encountered, the address of the next instruction is not usually known until after the execution phase of the jump instruction. In this case, the pipeline must be emptied and restarted with the next instruction when its address is known.

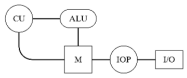

Figure 1-3 Adding an I/O processor to overlap I/O with CPU operations.

Jord-01ff.fm Page 4 Wednesday, July 24, 2002 8:20 PM

Instruction fetch	I1	I2	I3	Jump	–	–	–
Operand fetch	–	I1	I2	I3	Jump	–	–
Execute	–	–	I1	I2	I3	Jump	–
Result store	–	–	–	I1	I2	I3	Jump
Time	1	2	3	4	5	6	7

Figure 1-4 Overlapped fetch/execute cycle for an SISD computer.

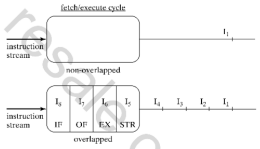

Figure 1-5 Comparison of SISD pipelined vs. non-pipelined fetch/execute cycle.

Several techniques to alleviate this problem and speed up the pipeline operation have been introduced, starting with an auxiliary instruction buffer introduced in the IBM system/370 model 165 [34,152,153]. The idea is to use two instruction buffers. Instructions are fetched into an instruction buffer sequentially addressed by the program counter. When a conditional jump is encountered, instructions will be fetched from the address of the target of the branch into the auxiliary instruction buffer. When the result is known, the buffers are switched if necessary.

The next major step of parallelism in sequential computers is the introduction of multiple arithmetic units in the CPU of the CDC 6600 [274]. In this situation, the CPU consists of several arithmetic units simultaneously capable of performing one operation each. To take advantage of this architecture, a local analysis of the instruction stream must be performed to determine which instructions in the stream can be processed simultaneously by the multiple arithmetic units. This *lookahead* analysis and its corresponding completion interlock, *scoreboarding*, was perhaps illustrated best in the CDC 6600 and its successors. The lookahead is used to prefetch instructions into an instruction buffer. The lookahead buffer is usually large enough to hold most scientific loops to optimize memory access. Scoreboarding is used to determine which instructions can be issued concurrently without a conflict. Conflicts may arise if two potentially concurrent instructions need to use the same arithmetic unit, *resource conflict*, or if they must store their results in the

Jord-01ff.fm Page 5 Wednesday, July 24, 2002 8:20 PM

same output register, *output dependence*. Conflicts also arise if an input register of one instruction is used as output of the other, *flow* and *antidependences*. An interlock and reservation logic guarantees correct execution of the instruction stream as was intended by the original program. Therefore, even though some instructions are executed out of the original sequence, the result is the same as if they had been executed in the given order. Thanks to technological advances, most of these architectural techniques are featured in desktop computers of today.

Compilers play a significant role in generating code that can take advantage of the parallelism embedded in these complex computer architectures. Consider evaluating the expression

$$EXP = A + B + C + (D \times E \times F) + G + H.$$

Using a left-to-right evaluation of the arithmetic expression, the corresponding tree and code that are generated by a compiler are shown in Figure 1-6. The tree represents the order in which the operations are performed. It represents the data dependence relationship among the operations that are needed to evaluate the expression. For example, according to the tree of Figure 1-6, operations $A + B$ must be completed before the execution of $T_1 + C$ at the next level can start. Operation $D \times E$ must be completed before $T_2 \times F$ but can be done at the same time as $A + B$. The height of this tree, the longest chain from the root to any leaf node, indicates that it will take a minimum of five steps to evaluate the expression assuming addition and multiplication take one time unit. The five steps is only possible if the computer system has the arithmetic units to perform one addition and one multiplication simultaneously. A smart compiler may generate a more parallel code using associativity and commutativity laws of add and multiply operations to reorder the evaluation of the expression. The resulting tree evaluation of height four is shown in Figure 1-7. The expression

can now be evaluated in four steps if the computer system can simultaneously perform two adds and one multiply. Arithmetic expression evaluation is further discussed in the next chapter.

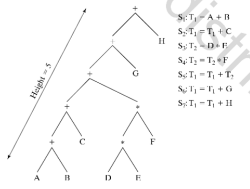

Figure 1-6 In-order tree traversal for evaluation of $A + B + C + (D \times E \times F) + G + H$.

Not for reproduction or distribution

Jord 01ff.fm Page 6 Wednesday, July 24, 2002 8:20 PM

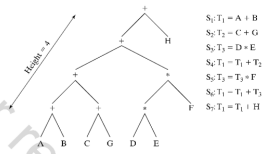

Figure 1-7 Commutativity and associativity applied to $A + B + C + (D \times E \times F) + G + H$.

These SISD parallelism techniques start with sequential code and do local reordering of operations using a combination of hardware and architecture specific code generation. This book does not specifically discuss instruction level parallelism (ILP), the parallelism that can be automatically extracted from sequential code and exploited by multiple arithmetic units in a very long instruction word (VLIW) or a superscalar machine. Our treatment is limited to explicitly parallel languages and machines. Of course, many of the fundamentals of parallel processing also apply to ILP, especially some of parallel algorithm characteristics discussed in Chapter 2 and some of the data dependence analysis framework established in Chapter 7, but we leave the direct treatment of ILP to other books.

1.1.2 Vector or SIMD Computers

The simplest step from sequential programs into the field of parallelism is to recognize that many inner loops are only used to specify that the same operations are to be performed repeatedly on a set of disjoint data items. These can be formulated as vector operations where the same operation is performed on each element of a vector. For example, a single vector add operation is used to produce a new vector, $C = A + B$, whose elements are component wise addition of two vectors, A and B. This single vector operation is equivalent to writing a sequential loop that adds corresponding elements of arrays A and B and assigns results to C one at a time.

The same operation can be carried out on multiple data items, that is, a vector operation, by a single control unit and replicated complete arithmetic units as in Figure 1-8(a). It can also be done by organizing the parallel hardware into a pipeline and processing the multiple data items in an assembly line fashion as shown in Figure 1-8(b). We will refer to the replicated,

Jord 01ff.fm Page 7 Wednesday, July 24, 2002 8:20 PM

non-pipelined architectures of Figure 1-8(a) as "true" SIMD to distinguish them from the pipelined SIMD architectures. True SIMD architectures can be configured so that each arithmetic unit has access to its own private memory. In this configuration, the arithmetic units are connected to each other through an interconnection network so that they can exchange data with each other. Another possible configuration in true SIMD machines is to let the arithmetic units share the memory. In this model, an interconnection (alignment network) is needed between the memory modules and the arithmetic units so that any arithmetic unit can access data in any memory module.

In this discussion, the categories SISD, SIMD, and MIMD are used from the instruction set point of view without regard to whether multiple operations are carried out on separate complete processors or whether the "parallel" operations are processed by a pipeline. Pipelining is another method for introducing parallelism into the operation of a computer and, as we have seen, is an important topic in von Neumann computer architecture. We take the view that, independent of pipelining, a machine in which a single instruction specifies operations on several data items is an SIMD machine. The advantage of this point of view is that pipelining then becomes an independent architectural variable that may be combined with Flynn's categories in an orthogonal manner. Data operations in the SISD case do not become parallel until pipelining is applied; the simple technique of fetch/execute overlap is one application of pipelining to SISD computers. One of the earliest non-pipelined SIMD computers, which was designed for vector and matrix processing, was the Illiac IV computer [38]. A machine with a very similar instruction set architecture, but which uses pipelined arithmetic units to support vector operations, is the Cray-1 [76], which we call a pipelined SIMD computer.

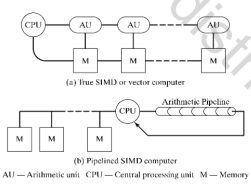

Figure 1-8 True and pipelined SIMD architectures.

OneNote https://onenote.officeapps.live.com/o/onenoteframe.aspx?ui...

Jord-01ff.fm Page 8 Wednesday, July 24, 2002 8:20 PM

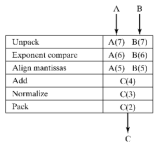

Figure 1-9 Example of an SIMD floating point addition pipeline.

In the true SIMD model, arithmetic units, also called PE for processing elements, perform the same operation, say floating point add, on a pair of vector elements that they are holding at the same time. Therefore, in the time it takes to do one addition, as many additions as there are arithmetic units are performed. Pipelining of arithmetic operations divides one operation, such as floating point addition, into several smaller functions and executes these subfunctions in parallel on different data. A snapshot of a floating point addition pipeline at a single pipeline minor cycle is shown in Figure 1-9. Pipelining can keep the amount of parallel activity high while significantly reducing the hardware requirement. Because the number of items inserted into the front end of the pipeline is not constrained by its length, pipelining also avoids the disadvantage of having vectors whose lengths are not even multiples of the number of arithmetic units in the true SIMD case. The vector pipeline usually empties after each vector operation and fills when a new vector operation begins; therefore, every new vector operation has a start-up cost associated with it.

True SIMD architectures have been used in numerous machines of a more special-purpose nature. General SIMD computers that have seen widespread commercial use have tended to be of the pipelined variety. Of course, no commercial machine uses a single form of parallelism. Multiple pipelines are used to enhance speed, and scalar arithmetic units are overlapped. The move from true to pipelined SIMD machines is dictated by an improved cost/performance ratio along with added flexibility in vector length. The numerous arithmetic units of the true SIMD computer are only partially used at any instant for short vectors. Chapter 3 provides a detailed treatment of SIMD computers.

1.1.3 Multiprocessors or MIMD Computers

Another way of introducing parallelism is to allow multiple instruction streams to be active simultaneously. This leads to the multiprocessor architectures. The analogy with organizing many people, each doing a small task to cooperatively perform a much larger task gives us some intuition about how to write programs for multiprocessors.

Jord-01ff.fm Page 9 Wednesday, July 24, 2002 8:20 PM

There are two prototypical forms of multiprocessor, distinguished by the way data are shared among processors. The shared memory form has a general interconnection network between multiple processors and multiple memories so that any processor can access any memory location. The distributed memory form associates a distinct memory with each processor with data being transmitted directly between processors. These two forms of MIMD computers or multiprocessors are shown in Figure 1-10(a).

The parallelism supported by multiprocessors may be used in two general ways. As each processor of the MIMD system is a complete CPU, each processor can execute a sequential program independently of the others; therefore, MIMD machines can execute multiple sequential programs in parallel for increased throughput. More interesting is to cause multiple processors to execute different parts of a single program to complete a single task faster. In this case, multiple processors will cooperatively and concurrently execute a parallel program. The cooperation and exchange of information among processors in shared memory MIMD is done through the read/write of shared memory locations by processors of a given task and through the use of synchronizations to control their access to shared data and the rate of progress of processes. In the distributed MIMD model, this is done by communication among processors through sending and receiving messages.

Successful systems of both the shared memory form and the distributed memory form have been built. Shared memory machines with tens of processors and distributed memory machines with hundreds of processors are commercially available. Figure 1-10(b) shows the configuration of an MIMD machine in which multiple instruction streams are supported by pipelining rather than by separate complete processors. The application of pipelining to multiple instruction stream execution can have the advantages of reduced hardware and increased flexibility in the number of instruction streams—the same advantages as were seen in the SIMD case. Because multiple, independent memory requests are generated by the parallel instruction streams, pipelining can also be applied effectively to the memory access path. Current terminology calls pipelined MIMD computers *multithreaded* machines.

In the SISD pipelined fetch/execute cycle of Figure 1-4, instructions that are in the pipeline come from a single instruction stream belonging to the process currently running. Dependences between instructions may introduce bubbles (null operations) into the execution pipeline. For example, if an instruction, *I*, needs the result of another instruction, *J*, ahead of it in the pipeline, then *I* cannot be issued until the result it needs is produced by *J* completing its execution. Figure 1-11 shows a snapshot of the operation of a multiprocessor pipeline. The instructions that are in the pipeline simultaneously come from different processes (instruction streams) and, therefore, do not (usually) need to wait on each other's completion. An explicit synchronization between instruction streams (because of communication between concurrent processes) may cause one process to be unable to proceed, in which case a null operation might proceed through the pipeline in the slot for that process until the delay condition is removed. Shared memory and distributed memory computers are presented in detail in Chapter 4 and Chapter 5, respectively.

Jord-01ff.fm Page 10 Wednesday, July 24, 2002 8:20 PM

Not for reproduction

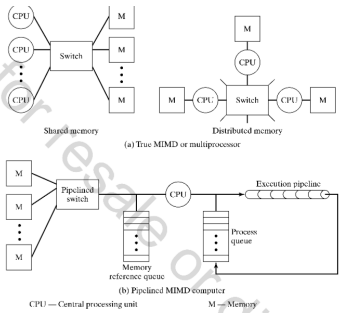

Figure 1-10 True and pipelined MIMD architectures.

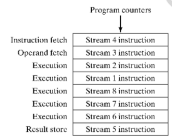

Figure 1-11 Pipelined fetch/execute cycle in shared-memory MIMD.

Not for resale or distribution

Jord-01ff.fm Page 11 Wednesday, July 24, 2002 8:20 PM

It is possible to discard the notion of flow of control entirely and take the attitude that it is only present as a vehicle for specifying how new data items depend on previous ones. This leads to a time-independent specification of data dependences and the complete elimination of the idea of control flow. The only remaining flow concept is the idea of data passing through, and being transformed by, operations that are successive in the sense that the input of one depends on the output of a "previous" operation. This idea is often denoted "data flow" and can be applied at the algorithmic, language design, or architectural level. The four categories of computer architectures defined by Flynn do not seem suitable for data flow machines, and we postpone their discussion until Chapter 7.

The increase in speed as a result of enhanced parallelism has been the province of computer architecture beginning with the move from serial by bit to parallel by word operation, moving through processor and I/O overlap and now focusing on the area of concurrent data operations. With the exception of the data flow idea mentioned above, one can almost trace a straight line evolution of the original von Neumann architecture into the parallel processors of today.

1.2 Interconnection Networks

An implication in all types of parallel architectures presented here is some form of connectivity between their components to facilitate the routing of data for the purpose of communication and cooperation between the parallel entities. This connectivity is provided through what is known as an interconnection network in the system.

In true SIMD computers, arithmetic units use the interconnection network to route data among themselves to get the right data to the processing elements needing them. In pipelined SIMD, an interconnection network enables parallel access to vector components stored in different memory modules. In shared memory MIMD, the interconnection network provides the processors access to the shared memory. Interconnection networks furnish the connectivity and the means for communication between processors of a distributed MIMD system. The implied interconnection networks in MIMD architectures are identified as "switch" in Figure 1-10.

An interconnection network is an important part of the parallel computer architecture. Its topology and structure directly influence the overall capability and performance of the system. A network structure can range from a simple bus structure connecting parallel processors to a complex, highly concurrent multistage network capable of performing all possible permutations of its inputs. Parallel machine performance is directly coupled with the degree of concurrency supplied by the interconnection network. To obtain high performance in a parallel system, the parallel entities must be able to communicate with each other concurrently. The higher the degree of concurrency in the interconnection network, the more concurrency is achieved by parallel units in a given system. In fact, concurrency in other units is wasted without a high concurrency network connecting them. With high concurrency come, however, design and cost complexities. While a fully interconnected network connects every processor to every other processor in an N-processor system, a bus interconnection allows all processors to connect to a single bus (Figure 1-12). The simple, low-cost bus allows only one communication at a time, while the fully connected network requires $N(N-1)/2$ bidirectional links and provides maximum concurrency at a cost that may be impractically high.

Not for resale or distribution

Jord-01ff.fm Page 12 Wednesday, July 24, 2002 8:20 PM

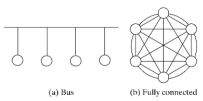

Figure 1-12 Two possible interconnection networks.

As some form of an interconnection network is needed by all parallel architectures that we will study, for the next four chapters we will assume its existence and will refer to it without too many details. We will study networks in detail in Chapter 6 once we have a clear understanding of the SIMD and MIMD computers and the need and importance of the role of interconnection networks in each.

1.3 Application of Architectural Parallelism

Applications programs for machines falling into different architectural categories require different structuring of solution algorithms for efficient operation. Put differently, given a program with a particular structure, the ways in which architectural parallelism can be applied are differ-

Not for resale or distribution

machine take full advantage of a pipelined SISD machine. Programs for a SIMD machine can be written in one of a number of "vector processing" languages. On the other hand, it is possible to apply an SIMD machine to an already written, sequential program using an automatic vectorizing compiler that takes as input a sequential program written in a specific language and produces vector parallel code for the given SIMD machine. The degree of success of an automatic vectorizing compiler depends on program structure. MIMD machines can execute multiple sequential programs in parallel for increased throughput or language extensions can be used to cause multiple processors to execute different parts of a program written in a sequential language. Work on automatic parallelizing compilers to speed up the execution of a sequential program using an MIMD architecture is more recent than work on vectorizing compilers.

All forms of architectural parallelism are applicable to the many large problems arising out of numerical linear algebra, but SIMD machines are tailored to them and can achieve extremely high performance in operations on long enough vectors. It is easy to partition the components of a vector over different instruction streams, but the flexibility afforded by multiple streams is not needed, and synchronization overhead may be excessive if parallelism is restricted to the level of single vector operations. Vector processing may be performed quite efficiently with respect to other work on an SISD computer as a result of predictable lookahead characteristics, but again the architecture is not specialized to it. The number of linear algebraic applications, the often large

Not for Distribution

Jord 01ff.fm Page 13 Wednesday, July 24, 2002 8:20 PM

size of these problems, and the considerable algorithm research that has taken place in this field, make this type of application an important standard against which to measure performance of any parallel architecture.

In numerous applications, the opportunity for parallel data operations is evident, but the operations differ from datum to datum, or there is enough irregularity in the structure of the data to make SIMD processing difficult or impossible. In these applications, one must either be content with pipelined SISD processing or move into the MIMD environment. Where multiple data items have irregular or conditional access patterns, MIMD processing may be preferred even when vectors are involved, as in the case of sparse matrix operations. At a higher level of algorithm structure is functional parallelism, in which large modules of the program operate in parallel and share data only in specific, easily synchronized ways.

1.4 Getting Started in SIMD and MIMD Programming

To start discussing the issues involved in parallel processing we need a way to show how a parallel algorithm might be executed by a given parallel processor; therefore, we need a parallel programming language. We really want a form of pseudocode that is not very machine-specific but reflects the features of the parallel computer architecture that are important for an understanding of the way operations are to be done simultaneously. We will start by characterizing, at the pseudocode level, the two types of parallel architecture that are closest to the traditional von Neumann computer, the SIMD and the MIMD.

We start with pseudocode for a traditional (SISD) computer that is similar to Pascal or C. We need assignment statements and a few control structures, as well as comments, to write readable pseudocode programs. Our initial constructs and the notation for them are specified by example in Table 1-1. Note that counted loops are more important to us than while loops, for example, because of the importance of an index in selecting from multiple data. We use the conventional mathematical relational operators, >, <, =, etc., for relational operations. To distinguish the relational "equal to," =, from an assignment statement we use ":=" for assignment. Because no modern language uses two key strokes for the ubiquitous assignment operator, this has the added advantage that the reader can easily distinguish pseudocode from actual programming language examples in later chapters.

As already mentioned, the SIMD mode of operation implies that the same operation is to be carried out simultaneously on multiple data items. The two ways of accomplishing this, with multiple arithmetic units or by means of pipelining, differ very little at the instruction set level. A single instruction must specify an operation and a collection of data items on which it is to be performed. (Two collections must be specified in the case of a two operand operation.) The simplest way of specifying a collection of data items of the same type, with the ability to select items using an index, is through the use of a vector. Hence, SIMD machines are often called vector computers. Either a multiple arithmetic unit or a pipelined SIMD machine can, therefore, be characterized by some way of specifying arithmetic operations applying to full vectors of data.

Not for Distribution

Jord 01ff.fm Page 14 Wednesday, July 24, 2002 8:20 PM

Table 1-1 Some pseudocode conventions.

Assignment	:=
End of statement	;
Statement grouping	**begin** ... **end** or { ... }
Statement label	LABEL:
Counted loops	**for** i := 0 **step** s **until** m ⟨statement⟩;
While loops	**while** (⟨condition⟩) ⟨statement⟩;
Conditional	**if** (⟨condition⟩) **then** ⟨statement⟩ **else** ⟨statement⟩;
Comments	/* Comments. */
Exit a structured block	**break**
Procedure	**procedure** ⟨name⟩ (⟨parameter list⟩) ⟨variable declaration⟩; ⟨statement⟩; **return**; **end procedure**
Procedure invocation	**call** ⟨procedure name⟩ (⟨parameter list⟩);

To extend the serial processor pseudocode so that it can be used to describe programs for vector processors, we want a generalized method of specifying vector arithmetic. We do it by introducing a vector assignment statement of the form

```
⟨indexed variable⟩ := ⟨indexed expression⟩, (⟨index range⟩);
```

The indexed variable on the left side of the assignment must depend on the index appearing in the parenthesized index range expression in such a way that the individual assignments that are made in parallel are all to different storage locations. An example might be

```
C[i,j] := C[i,j] + A[i,k]*B[k,j], (0 ≤ j ≤ N-1);
```

where assignments are made in parallel for different values of j, and, hence, to elements in different columns of the matrix C. With this simple extension, we can describe a matrix multiply as it would be performed on a SIMD computer as shown in Program 1-1.

In reading the pseudocode, it can be seen that there are two statements that specify parallel activity: one to initialize inner product sums to zero and the other to add the next inner product term. The second parallel statement would involve several machine instructions to do the two floating-point operations and to evaluate subscript expressions for the two dimensional arrays. If sufficient hardware units are available, either arithmetic units or pipeline stages, up to N operations may be performed in parallel. If only $M < N$ operations can be specified by a single machine instruction, the compiler will also have to generate code to sequence through the N vector elements, M at a time. Thus, the pseudocode is really at a higher level than the machine instruction set even though our SIMD parallel extension seems fairly low level. The fact is that, because SIMD parallelism occurs at the level of a single operation, it is necessarily reflected at a low level of program structure.

Not for Distribution

OneNote https://onenote.officeapps.live.com/o/onenoteframe.aspx?ui...

Jord01ff.fm Page 15 Wednesday, July 24, 2002 8:20 PM


```
/* Matrix multiply, C := A · B.  Compute elements of C by

   c_ij = Σ_{k=0}^{N-1} a_ik b_kj        */

for i := 0 step 1 until N-1
   begin /* Compute one row of C. */
      /* Initialize the sums for each element of a row of C. */
      C[i, j] := 0, (0 ≤ j ≤ N-1);
      /* Loop over the terms of the inner product. */
      for k := 0 step 1 until N-1
         /* Add the kth inner product term across columns in parallel. */
         C[i, j] := C[i, j] + A[i, k]*B[k, j], (0 ≤ j ≤ N-1);
      /* End of product term loop. */
   end /* of all rows. */
```

Program 1-1 Matrix multiply pseudocode for an SIMD computer.

To characterize the operation of MIMD computers at a fairly machine-independent level, we need to extend the SISD pseudocode in a different way. Again, there is no difference between multiple processor and pipelined implementations. In either case, there are multiple processes that are capable of executing different code simultaneously. We will think of the code for a multiprocessor as constituting one program that is executed by all processes. Of course, each processor may be executing at a different place in the code, or even in a different procedure.

A note on careful use of terminology is important to keep the discussion from becoming confusing. The component of a computer that can run a program is called a *processor*. Because many processors can be running the same program simultaneously, we need to name the dynamic sequence of operations being performed by a specific processor so as to distinguish it from the static body of text that constitutes the *program*. We call this dynamic execution sequence a *process*. This is a fairly general use of the term *process*. Many authors use it to designate something more specific. Other terms that are sometimes used to describe what we call a process are an *instruction stream*, a *task*, or a *thread of execution*. In summary, our notation is

- processor—a physical unit capable of executing a program;
- process—the dynamic sequence of operations executed by a processor;
- program—a static body of code specifying a sequence of operations.

When multiple processes are used to solve a single problem, there must be a way for data computed by one process to be made available for use by a different process. Initially, the simplest way to provide such a capability is to think of all processes as sharing access to the same memory. It also must be possible for one process to exercise some control over the progress of another. It should at least be possible to start a new process executing at a specific place in the

Jord01ff.fm Page 16 Wednesday, July 24, 2002 8:20 PM

program and to determine whether it has completed a specified sequence of operations. A simple pair of operations to accomplish this control consists of **fork** and **join**. The fork operation takes a single argument that specifies a label in the program at which the newly started process will begin execution while the original process continues with the statement following the fork. The join operation takes an integer argument that specifies how many processes are to participate in the join. All processes but one that execute the join will be terminated, and that one "original" process will proceed only after the specified number have all executed **join**.

Another necessary pseudocode statement is declarative in nature. There should be a way to specify temporary variables that refer to a different storage location for each process. The corresponding feature in SIMD is handled by the index that appears in the index range expression. This index serves to separate different items of the multiple data. In MIMD it is natural to specify that a single variable name represents a different memory cell for each process. This can be handled by declaring the parallelism type of a variable, or more accurately its parallel storage class, as **private**. This means that each process has its own private copy of the variable referred to by that name. Although it is possible to have variables that are shared by some processes but not others, we start with the simple distinction between variables that are uniformly shared by all processes and those that are strictly private to a single process. A **shared** declaration means that the declared variables are shared by all processes.

A summary of our pseudocode extensions for describing multiprocessor algorithms is

fork ⟨label⟩	Start a process executing at ⟨label⟩;
join ⟨integer⟩	Join ⟨integer⟩ processes into one;
shared ⟨variable list⟩	Make the storage class of the variables shared.
private ⟨variable list⟩	Make the storage class of the variables private.

With these extensions it is possible to write pseudocode that gives a general understanding of the structure of a multiprocessor program without being specific to one machine or a limited class of machines. The matrix multiply program is written for a shared memory MIMD machine in Program 1-2.

In the multiprocessor pseudocode, it is somewhat difficult to see all the parallelism that exists in the program. The only explicit parallel extensions that appear are one occurrence each of **fork**, **join**, **shared**, and **private**. In reading the section of code between the label DOCOL and the **join** statement, it must be kept implicitly in mind that N processes, each with a different value of j, are executing the code simultaneously, although they are not necessarily executing the same statement at exactly the same time. MIMD parallelism can, thus, be thought of as asynchronous parallelism, whereas SIMD parallelism is synchronous at the statement level, with all the arithmetic for a parallel statement being complete before the next statement is executed.

1.5 Parallelism in Algorithms

One way to detect parallelism in a sequential algorithm is to look for operations that can be carried out independently of each other. The question then is how to identify the independence of operations.

Jord01ff.fm Page 17 Wednesday, July 24, 2002 8:20 PM


```
/* Matrix multiply, C := A · B.  Compute elements of C by
```

Not for Distribution

/* Matrix multiply, C := A * B. Compute elements of C by

$$c_{ij} = \sum_{0}^{N-1} a_{ik} b_{kj} \quad */$$

```
private i, j, k;
shared A[N,N], B[N,N], C[N,N], N;
/* Start N-1 new processes, each for a different column of C. */
for j := 0 step 1 until N-2 fork DOCOL;
/* The original process reaches this point and does column N-1. */
j := N-1;
DOCOL: /* Executed by N processes, each doing one column of C. */
for i := 0 step 1 until N-1
   begin /* Compute a row element of C. */
      /* Initialize the sum for the inner product. */
      C[i, j] := 0;
      /* Loop over the terms of the inner product. */
      for k := 0 step 1 until N-1
         /* Add the kth inner product term. */
         C[i, j] := C[i, j] + A[i, k]*B[k, j];
      /* End of product term loop. */
   end /* of all rows. */
join N;
```

Program 1-2 Matrix multiply pseudocode for a multiprocessor.

In general, we define three types of dependences between operations, namely, output, flow, and antidependences. Consider the sequence of statements in a sequential algorithm. For each statement, S_i, define the set of all variables read by S_i as its input set, $I(S_i)$, and the set of all variables written by execution of S_i as its output set, $O(S_i)$. Then two statements S_i and S_j are independent of each other if all three conditions of Eq. (1.1) hold.

$$\begin{aligned} &1.\ O(S_i) \bigcap O(S_j) = \Phi \quad \text{output independence} \\ &2.\ I(S_j) \bigcap O(S_j) = \Phi \quad \text{flow independence} \\ &3.\ I(S_j) \bigcap O(S_j) = \Phi \quad \text{anti independence} \end{aligned} \tag{1.1}$$

In other words, no two statements write to the same variables, and no input variable of a statement is an output variable of another. The conditions stated in Eq. (1.1) are known as Bernstein's conditions [47].

The statements in Eq. (1.1) may be considered at various levels of granularity. For example, at the level of multiple arithmetic units of an SISD processor as in the CDC 6600 example,

Jordi 01ff.fm Page 18 Wednesday, July 24, 2002 8:20 PM

the lookahead buffer and scoreboard essentially detect parallelism in the machine level instruction sequences using these conditions. At a higher level, one might consider the statements S_i and S_j as large-code segments or even entire procedures and detect dependence among pairs of procedures. It is important to note that parallelism is commutative but not transitive. In other words, if S_i is independent of S_j, and S_j is independent of S_k, S_i and S_k are not necessarily independent. Therefore, all pairs of statements that are being considered for concurrent execution must be tested for independence.

Compilers can use the conditions of Eq. (1.1) to generate parallel code from sequential programs. This approach of detecting parallelism in a sequential algorithm often does not result in the best parallel program and implementation. Most often, restructuring of operations must be done to reveal the hidden parallelism in the computation. In general, the type of parallelism we look for depends on the type of parallel architecture that implements the task at hand, and except in very simple cases, new parallel algorithms must be designed. For example, in the SIMD pseudocode for matrix multiply of Program 1-1, we have chosen the ordering of the three nested loops over i, k, and j indices that is different from the usual sequential algorithm order of i, j, k, respectively. In the typical sequential implementation order, the computation to produce the result matrix C is carried out by completing the calculation for $C[1, 1]$ before proceeding to calculate $C[1, 2]$ and so on. This sequential order performs the dot product of row i of matrix A and column j of matrix B to produce element $C[i, j]$. The component wise multiplication of row i of matrix A and column j of matrix B is an efficient vector operation for an SIMD type architecture. However, the dot product of the two vectors requires the summation of the product vector into a single element, known as a *reduction* operation. This reduction operation is not performed efficiently in most SIMD architectures. In the true SIMD case, for example, the arithmetic units must communicate via the interconnection network to do a pairwise summation of the product vector. Sequentially, the summation of N numbers can be computed in a loop by adding one number to the sum in each iteration. The summation can be done in its most parallel form if disjoint pairs of operands are added together first. Then disjoint pairs of partial sums are added next and so on until the single final sum is produced. The evaluation tree for this parallel summation will be a full binary tree if N is a power of 2. The pairwise summation of N numbers takes $\lceil \log_2 N \rceil$ steps even at the computer hardware level. Recall that the notation, $\lceil x \rceil$ denotes x if x is an integer and the next integer greater than x if x has a fractional part. In the SIMD version of the matrix multiply, we have reordered the operations to produce the elements of the C matrix one row at a time. This way the underlying architecture efficiently performs a scalar times vector operation and adds two vectors without the need for a reduction operation.

In the SIMD case, the attempt has been to identify efficient parallelism at the lowest machine instruction level. For the MIMD architecture, we applied parallelism at the highest possible level by partitioning the work so that each processor is assigned the largest amount of work to be done. All three algorithms for the matrix multiply do the same number of operations but in different orders. We will study parallel algorithms for various architectures and their properties in more depth in subsequent chapters.

Jordi 01ff.fm Page 19 Wednesday, July 24, 2002 8:20 PM

1.6 Conclusion

A review of the evolution of sequential computers has revealed the many ways parallelism has been introduced into sequential computers over time. While the application of parallelism in sequential machines has been hidden from user programs, it has essentially been the key mechanism of delivering speed in computers from the architecture and system design perspective as opposed to advances in solid-state electronic technology. Parallel applications and architectures are, thus, the next natural step in our quest toward computational speed that can accompany and further the progress in the technology.

In this introductory chapter, we have seen two important architectures for parallel computation, the SIMD architecture that applies the same operation to multiple data items simultaneously and the MIMD architecture that executes multiple instruction streams simultaneously, applying multiple independent instructions to multiple data values in parallel. A brief sketch of programming for these two architectures showed that the foremost issue in SIMD programming is that of specifying operations on vectors at the statement level, while the paramount concern in MIMD programming is creating and coordinating multiple sequential instruction streams that share data.

Parallel algorithms are designed to take advantage of specific architectures. A sequential algorithm may have to be restructured to detect parallel operations. On the other hand, as we will see in the next chapter, completely new algorithms may have to be designed to perform the same task with various levels of parallelism.

The fundamentals of parallel processing, therefore, involve architectures, programming languages, and algorithms. The next four chapters consider major architectural styles and the

languages and program types that go with them. Chapter 3 treats SIMD architectures and the vector-oriented algorithms and languages suited to them. Chapter 4 and Chapter 5 treat MIMD architectures in which processors do and do not share memory, respectively. Chapter 6 treats the interconnection networks that enable communication among processing elements and memories in both SIMD and MIMD machines. Chapter 7 examines the fundamental impact of data dependence on parallel execution, both in the context of extracting parallelism from sequential programs and in using data flow representations to avoid introducing artificial parallelism into programs when they are written.

1.7 Bibliographic Notes

The foundations of overlap processing in sequential machines are discussed in Baer's text [34], incorporating original parallel instruction scheduling work from the CDC 6600 scoreboard [274] and Tomasulo's algorithm [276]. A good survey of parallel architectures can be found in [150]. It covers architectures that are important from the architectural point of view, even if many are no longer available. An early survey [243] covers pipelined architectures, while a slightly later book by Kogge [178] is a classic work on pipelined architectures with a good discussion of pipelined SIMD. A survey of interconnection networks can be found in [106]. A broad survey of parallel algorithms across distinct architecture classes can be found in [74]. It includes algorithms

Not for resale or distribution

Jord-01ff.fm Page 20 Wednesday, July 24, 2002 8:20 PM

for vector computers as well as for shared and distributed memory multiprocessors. The programming and design of vector computers is well treated in [144]. An integrated treatment of parallel architectures, algorithms, and languages is given in [15]. Trew and Wilson's book [278] is a nice survey of commercially available parallel computers in 1990. It demonstrates how quickly a list of current architectures becomes dated. It even has a section on machines from companies that had recently failed.

Problems

1.1 Write SIMD and MIMD pseudocode for the tree summation of n elements of a one-dimensional array for n a power of two. Do not use recursive code, but organize the computations as iterations. The key issue is to determine which elements are to be added by a single arithmetic unit or a single processor at any level of the tree. You may overwrite elements of the array to be summed.

1.2 What mathematical property do sum, product, maximum, and minimum have in common that allows them to be done in parallel using a tree-structured algorithm?

1.3 The SIMD matrix multiply pseudocode of Program 1-1 is written to avoid doing an explicit reduction operation that is needed for the dot product of two vectors. Write another version of SIMD matrix multiply pseudocode that avoids the reduction operation. Describe the order of operations and compare it with both the sequential version and the SIMD version of Program 1-1.

1.4 Apply Bernstein's conditions to the compiler codes generated for evaluation of expressions in Figure 1-6 and Figure 1-7. In each case, determine which statements are independent of each other and can be executed in parallel. Detect and identify the type of dependences for statements that are not independent. Explain what might happen if two dependent statements are executed concurrently.

1.5 To apply Bernstein's conditions to the statements of Figure 1-6 to determine the independence of operations between the statements, how many pairs of statements must be examined? How many conditions must be tested in general for a code consisting of N statements?

1.6 Assume each stage of the floating addition pipeline of Figure 1-9 takes one time unit. Compare the performance of this pipelined floating point add with a true SIMD machine with six arithmetic units in which a floating point addition takes six time units. Show how long it takes to add two vectors of size 20 for both true and pipelined SIMD.

1.7 Consider the execution of the sequential code segment

```
S1:   X = (B - A)(A + C)
S2:   Y = 2D(D + C)
S3:   Z = Z(X + Y)
S4:   C = E(F - E)
S5:   Y = Z + 2F - B
S6:   A = C + B/(X + 1)
```

(a) Write the shortest assembly language code using *add*, *sub*, *mul*, and *div* for addition, subtraction, multiplication, and divide, respectively. Assume an instruction format with register address field so that *R1 = R2 + R3* is equivalent to *add R1, R2, R3*. Assume there are as many registers as needed, and further assume that all operands have already been loaded into registers, therefore, ignoring memory reference operations such as load and store.

(b) Identify all the data dependences in part (a).

Jord-01ff.fm Page 21 Wednesday, July 24, 2002 8:20 PM

(c) Assume that add/sub takes one, multiply three, and divide 18 time units on this multiple arithmetic CPU, respectively, and that there are two adders, one multiplier, and one divide unit. If all instructions have been prefetched into a lookahead buffer and you can ignore the instruction issue time, what is the minimum execution time of this assembly code on this SISD computer?

Not for resale or distribution